

dataloader/kuaiREC.py (Q&A)

Q: What is the purpose of `dataloader/kuaiREC.py`? **A:** It defines the PyTorch dataset and dataloader wrappers for the preprocessed KuaiRec data. It is responsible for converting the stored pandas DataFrames (loaded from a pickle) into GPU/CPU tensors and returning batches in the “features dict + label” format expected by the models.

Q: What are the two main classes in this file? **A:** `KUAIRecDataset` is a `torch.utils.data.Dataset` that materializes all columns into tensors on a device; `KUAIRecDataLoader` is a lightweight wrapper that reads the pickled data, constructs `DataLoader` objects for each split (e.g. `train` and `test`), and exposes them via `__getitem__`.

Q: What does one sample look like (inputs and label)? **A:** `__getitem__` returns (`features_dict`, `label`) where `features_dict` maps feature names to tensors (shape depends on feature type), and `label` is `play_time` (a float scalar per row, normalized by preprocessing).

Q: How are feature dtypes decided? **A:** The `format` method uses the feature description (a list of tuples (name, size, type)) to cast feature tensors: sparse/sequence IDs are cast to `torch.long`, continuous/mask/other features to `torch.float32`, while the label is left as-is.

Q: What is a common usage pattern in training scripts? **A:** Run scripts create a `KUAIRecDataLoader` and then iterate over `dataloaders['train']` or `dataloaders['test']` to get batches.

```
dataloaders = KUAIRecDataLoader("kuaiREC", "/path/to/kuaiREC_data.pkl", device, bsz=2048)
for features, label in dataloaders["train"]:
    y = model(features)
```

Q: Full source code? **A:**

```
import numpy as np
import pandas as pd
import pickle
import os
import torch
from torch.utils.data import Dataset, DataLoader

class KUAIRecDataset(Dataset):
    """
    Load a KuaiRec Dataset
    """
    def __init__(self, dataset_name, df, description, device):
        super(KUAIRecDataset, self).__init__()
```

```

        self.dataset_name = dataset_name
        self.df = df
        self.length = len(df)
        self.name2array = {name: torch.from_numpy(np.array(list(df[name]))).reshape([self.
            length, -1])).to(device) \
            for name in df.columns}
        self.format(description, device)
        self.features = [name for name, size, type in description if type != 'label']
        self.label = 'play_time'

    def format(self, description, device):
        for name, size, type in description:
            if type == 'spr' or type == 'seq':
                self.name2array[name] = self.name2array[name].to(torch.long)
            elif type == 'ctn' or type == 'seqm' or type == 'other':
                self.name2array[name] = self.name2array[name].to(torch.float32)
            elif type == 'label':
                pass
            else:
                raise ValueError('unkwon_type_{}'.format(type))

    def __getitem__(self, index):
        return {name: self.name2array[name][index] for name in self.features}, \
            self.name2array[self.label][index].squeeze()

    def __len__(self):
        return self.length

class KUAIRECDataLoader(object):
    """
    Load KUAIRANK16DataLoader for torch train/eval

    :param dataset_path: dataset path
    """

    def __init__(self, dataset_name, dataset_path, device, bsz=32):
        assert os.path.exists(dataset_path), '{}_does_not_exist'.format(dataset_path)
        with open(dataset_path, 'rb+') as f:
            data = pickle.load(f)
        self.dataset_name = dataset_name
        self.dataloaders = {}
        self.description = data['description']
        for key, df in data.items():
            if key == 'description':
                continue
            self.dataloaders[key] = DataLoader(KUAIRECDataset(dataset_name, df, self.
                description, device), batch_size=bsz, shuffle=False)
        self.keys = list(self.dataloaders.keys())

    def __getitem__(self, name):
        assert name in self.keys, '{}_not_in_keys_of_datasets'.format(name)
        return self.dataloaders[name]

```
